

■ Python Language Lab

Complete Viva Voice Question & Answer Guide

Section	Content	Page
Section A	Syllabus-Based Viva Q&A (Topics 1–10)	2
Section B	Additional Teacher-Likely Questions	Later

■ Exam Tips: Always explain your code line by line. Mention time complexity where applicable. Know the differ

SECTION A: Syllabus-Based Viva Questions

Topic 1 — Basic Data Types in Python

Q: What are the basic data types in Python?

A: Python has: int (integers), float (decimal numbers), str (strings), bool (True/False), complex (complex numbers), list, tuple, set, dict, and NoneType.

Q: What is the difference between int and float?

A: int stores whole numbers (e.g., 5, -3). float stores decimal/fractional numbers (e.g., 3.14, -0.5). Floats take more memory than ints.

Q: How do you check the type of a variable?

A: Using the built-in type() function. Example: type(5) returns .

```
x = 3.14
print(type(x))    #
```

Q: What is type casting? Give an example.

A: Converting one data type into another. E.g., int('5') converts string '5' to integer 5. Functions: int(), float(), str(), bool().

Q: Is Python statically or dynamically typed?

A: Dynamically typed — you do not declare types; the type is determined at runtime based on the value assigned.

Topic 2 — Distance Between Two Points & Command-Line Arguments

Q: What formula is used to compute distance between two points?

A: Euclidean distance: $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$. In Python, use math.sqrt() or ** 0.5.

```
import math
d = math.sqrt((x2-x1)**2 + (y2-y1)**2)
```

Q: How do you take input from the user in Python?

A: Using input() function. It always returns a string, so cast it: x = float(input('Enter x: ')).

Q: What is sys.argv? How is it used in add.py?

A: sys.argv is a list in the sys module that holds command-line arguments. sys.argv[0] is the script name, sys.argv[1] onwards are passed arguments.

```
import sys
a = int(sys.argv[1])
b = int(sys.argv[2])
print(a + b)
```

Q: How do you run a Python script with command-line arguments?

A: From terminal: python add.py 5 10 — this passes 5 and 10 as arguments.

Topic 3 — Even/Odd Check & For Loop with Fractions

Q: How do you check if a number is even or odd in Python?

A: Using the modulo operator %. If $n \% 2 == 0$ then it is even, else odd.

```
n = int(input())
if n % 2 == 0:
    print('Even')
```

```
else:
    print('Odd')
```

Q: What is the modulo operator?

A: % returns the remainder of division. E.g., $7 \% 3 = 1$. It is commonly used to check divisibility.

Q: Write the program to print decimal equivalents of 1/2. 1/3. 1/10.

```
for i in range(2, 11):
    print(f'1/{i} = {1/i:.4f}')
```

Q: What is range()? How does range(2, 11) work?

A: range(start, stop) generates numbers from start up to (but NOT including) stop. range(2,11) gives 2,3,4,...,10.

Topic 4 — List, Tuple, For Loop, While Loop

Q: What is the difference between a list and a tuple?

A: List: mutable (can be changed), uses square brackets []. Tuple: immutable (cannot be changed after creation), uses parentheses (). Tuples are faster and used for fixed data.

```
my_list = [1, 2, 3]      # mutable
my_tuple = (1, 2, 3)     # immutable
```

Q: How do you loop over a sequence using a for loop?

A: for item in sequence: iterates over each element. Works on lists, tuples, strings, dictionaries, etc.

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

Q: What is the difference between for loop and while loop?

A: for loop: used when number of iterations is known (iterates over a sequence). while loop: used when iterations depend on a condition (runs as long as condition is True).

Q: Write a while loop countdown program.

```
n = int(input('Enter number: '))
while n >= 0:
    print(n)
    n -= 1
```

Q: What happens if you try to modify a tuple?

A: Python raises a TypeError: 'tuple' object does not support item assignment.

Topic 5 — Sum of Primes & Fibonacci Even Terms

Q: What is a prime number? How do you check if a number is prime?

A: A prime has no divisors other than 1 and itself. To check: try dividing n by all integers from 2 to sqrt(n). If none divides it evenly, it is prime.

```
import math
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(math.sqrt(n))+1):
        if n % i == 0: return False
    return True
```

Q: What is the Sieve of Eratosthenes?

A: An efficient algorithm to find all primes up to N. Create a boolean list, mark multiples of each prime as False starting from 2. The remaining True positions are primes. More efficient than checking each number individually.

Q: What is the Fibonacci sequence?

A: A sequence where each number is the sum of the two preceding ones: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144...

Q: How do you find the sum of even-valued Fibonacci terms not exceeding 4 million?

```
a, b, total = 0, 1, 0
while a <= 4000000:
    if a % 2 == 0:
        total += a
    a, b = b, a+b
print(total)
```

Q: Why do even Fibonacci numbers occur at every 3rd position?

A: Because odd+odd=even, even+odd=odd, odd+even=odd — the pattern E,O,O,E,O,O... repeats, so every 3rd Fibonacci number is even.

Topic 6 — Dictionary: Character Count & String Methods

Q: What is a dictionary in Python?

A: An unordered collection of key-value pairs. Keys must be unique and immutable. Values can be any type. Defined with curly braces {}.

```
d = {'name': 'Alice', 'age': 20}
```

Q: How do you count characters in a string and store in a dictionary?

```
s = 'hello world'
freq = {}
for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Q: What is dict.get(key, default)?

A: Returns the value for key if it exists; otherwise returns default. Avoids KeyError.

Q: Explain split() and join() methods.

A: split(sep): splits a string into a list at each separator. join(list): joins list elements into a string with the separator between them.

```
s = 'John 25 Jan'
parts = s.split() # ['John', '25', 'Jan']
result = '-'.join(parts) # 'John-25-Jan'
```

Q: How would you store a person's birthday using a dictionary?

```
birthday = {'name': 'Alice', 'day': 15, 'month': 'Aug', 'year': 2000}
print(f"{birthday['name']} born {birthday['day']} {birthday['month']}")
```

Topic 7 — Character Frequency & File Type Detection

Q: How do you count character frequency in a file?

```
freq = {}
with open('file.txt', 'r') as f:
    for ch in f.read():
        freq[ch] = freq.get(ch, 0) + 1
```

Q: How can you guess if a file is Python, C, or text using character frequency?

A: Python files: high frequency of '#', ':', 'def', 'import'. C files: high frequency of '{', '}', ';', '#include'.
Text files: high alphabetic frequency, spaces, periods — no special symbols.

Q: What is the difference between open() modes 'r', 'w', and 'a'?

A: 'r' = read (default), 'w' = write (overwrites), 'a' = append (adds to end). Use 'rb'/'wb' for binary files.

Q: Why should we use 'with open()' instead of just 'open()'?

A: 'with' automatically closes the file after the block, even if an exception occurs. This prevents resource leaks.

Topic 8 — File Operations: Reverse Lines, Count Stats

Q: How do you print each line of a file in reverse order?

```
with open('file.txt', 'r') as f:
    lines = f.readlines()
    for line in reversed(lines):
        print(line, end='')
```

Q: How do you count characters, words, and lines in a file?

```
with open('file.txt', 'r') as f:
    content = f.read()
    lines = content.split('\n')
    words = content.split()
    chars = len(content)
    print(f'Lines:{len(lines)} Words:{len(words)} Chars:{chars}')
```

Q: What is the difference between read(), readline(), and readlines()?

A: read(): reads entire file as one string. readline(): reads one line at a time. readlines(): reads all lines into a list.

Q: How do you reverse a string in Python?

A: Using slicing: s[::-1]. Or using reversed(): ''.join(reversed(s)).

Topic 9 — Nearly Equal Strings & One-Line GCD/LCM

Q: What does 'nearly equal' mean for strings? What is a single mutation?

A: Two strings a and b are nearly equal if b can be obtained from a by exactly one of: insert a character, delete a character, or replace a character (one edit distance of 1).

Q: Write the nearly_equal function.

```
def nearly_equal(a, b):
    if abs(len(a)-len(b)) > 1: return False
    if len(a) == len(b):
        return sum(x!=y for x,y in zip(a,b)) == 1
    if len(a) < len(b): a, b = b, a
    return any(a[:i]+a[i+1:]==b for i in range(len(a)))
```

Q: Write a one-line GCD function.

```
import math
gcd = lambda a, b: math.gcd(a, b)
```

Q: Write a one-line LCM function.

```
lcm = lambda a, b: a * b // math.gcd(a, b)
```

Q: What is GCD and LCM?

A: GCD (Greatest Common Divisor): largest number that divides both a and b. LCM (Least Common Multiple): smallest number divisible by both a and b. LCM(a,b) = a*b / GCD(a,b).

Q: What is a lambda function?

A: An anonymous, single-expression function. Syntax: lambda args: expression. Good for short, one-time use functions.

Topic 10 — Sorting Algorithms: Merge Sort, Selection Sort, Insertion Sort

Q: Explain Merge Sort. What is its time complexity?

A: Merge Sort is a divide-and-conquer algorithm. It splits the array in half recursively until single elements, then merges sorted halves. Time: $O(n \log n)$ for all cases. Space: $O(n)$.

Q: Write the Merge Sort algorithm.

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(L, R):
    result, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i+=1
        else: result.append(R[j]); j+=1
    return result + L[i:] + R[j:]
```

Q: Explain Selection Sort.

A: Find the minimum element in the unsorted portion and swap it to the front. Repeat for remaining elements. Time: $O(n^2)$ always.

Q: Write Selection Sort.

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_i = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_i]: min_i = j
        arr[i], arr[min_i] = arr[min_i], arr[i]
    return arr
```

Q: Explain Insertion Sort.

A: Pick each element and insert it into its correct position among the already-sorted elements. Like sorting playing cards in hand. Best case $O(n)$, worst case $O(n^2)$.

Q: Write Insertion Sort.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Q: Compare the three sorting algorithms.

Algorithm	Best	Average	Worst	Space	Stable?
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

SECTION B: Additional Questions Teachers Are Likely to Ask

These questions go beyond the syllabus but are commonly asked to check understanding.

B1 — Python Fundamentals (General)

Q: What is Python? Who created it?

A: Python is a high-level, interpreted, general-purpose programming language. Created by Guido van Rossum, first released in 1991.

Q: What is PEP 8?

A: PEP 8 is Python's official style guide. It recommends: 4 spaces for indentation, snake_case for variable names, UPPER_CASE for constants, max line length of 79 characters.

Q: What is the difference between == and is?

A: == compares values (are they equal?). is compares identity (do they point to the same object in memory?). Example: [1,2] == [1,2] is True but [1,2] is [1,2] is False.

Q: What are mutable and immutable objects?

A: Mutable: can be changed after creation — list, dict, set. Immutable: cannot be changed — int, float, str, tuple. Immutable objects are safer to use as dictionary keys.

Q: What is the difference between append() and extend() in a list?

A: append(x) adds x as a single element. extend([x,y]) adds each element of the iterable individually.

```
a = [1,2]
a.append([3,4])    # [1, 2, [3, 4]]
a.extend([3,4])    # [1, 2, 3, 4]
```

Q: What is *args and **kwargs?

A: *args allows a function to accept any number of positional arguments (stored as a tuple). **kwargs accepts any number of keyword arguments (stored as a dict).

Q: What is a list comprehension?

A: A concise way to create lists. Syntax: [expression for item in iterable if condition].

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

B2 — Functions, Scope, and OOP

Q: What is the difference between local and global variables?

A: Local variables are defined inside a function and accessible only there. Global variables are defined outside all functions. Use the global keyword inside a function to modify a global variable.

Q: What is recursion? Give an example.

A: A function calling itself. Must have a base case to stop. Example: factorial.

```
def factorial(n):
    if n == 0: return 1
    return n * factorial(n-1)
```

Q: What are *args in the context of recursion vs iteration?

A: Recursion uses the call stack (can hit stack overflow for large inputs). Iteration uses a loop (more memory-efficient for large inputs). Fibonacci: recursive $O(2^n)$ vs iterative $O(n)$.

Q: What is a class and an object?

A: A class is a blueprint/template. An object is an instance of a class. Example: class Dog: defines a blueprint; my_dog = Dog() creates an object.

Q: What is __init__ in Python?

A: The constructor method, called automatically when an object is created. Used to initialise object attributes.

Q: Explain inheritance in Python.

A: A child class inherits attributes and methods from a parent class. Use: `class Child(Parent)`. Promotes code reuse. Use `super()` to call the parent's constructor.

B3 — Exception Handling & Modules

Q: What is exception handling? Why is it important?

A: Handling runtime errors gracefully using try-except blocks so the program doesn't crash. Important for file operations, user input, network calls, etc.

```
try:
    x = int(input())
except ValueError:
    print('Not a valid integer')
finally:
    print('Done')
```

Q: What is the difference between syntax error and runtime error?

A: Syntax error: code violates Python grammar rules — caught before execution (e.g., missing colon). Runtime error (exception): occurs during execution (e.g., `ZeroDivisionError`, `FileNotFoundError`).

Q: What is the 'finally' block?

A: Code in finally always executes whether an exception occurred or not. Typically used to release resources (close files, DB connections).

Q: What is the math module? Name 5 functions from it.

A: Built-in module for mathematical operations. Functions: `math.sqrt()`, `math.floor()`, `math.ceil()`, `math.pow()`, `math.gcd()`, `math.pi` (constant), `math.factorial()`.

Q: What is the difference between `import math` and `from math import sqrt`?

A: `import math`: imports the whole module; access with `math.sqrt()`. `from math import sqrt`: imports only `sqrt`; call directly as `sqrt()`. The latter avoids namespace clutter.

B4 — Strings & File Handling Deep Dive

Q: Name 8 important string methods with examples.

A: `upper()`, `lower()`, `strip()` (removes whitespace), `replace(old,new)`, `find(sub)`, `count(sub)`, `startswith(s)`, `endswith(s)`, `split(sep)`, `join(list)`, `isdigit()`, `isalpha()`.

Q: What is string formatting? Explain f-strings.

A: f-strings (formatted string literals, Python 3.6+) allow embedding expressions inside strings with `{}`. Example: `f'Hello {name}, you are {age} years old.'`

Q: What is the difference between text mode and binary mode for files?

A: Text mode (`'r'`, `'w'`): reads/writes strings, handles newline conversion (`\n`). Binary mode (`'rb'`, `'wb'`): reads/writes raw bytes, no conversion — used for images, PDFs, executables.

Q: How do you handle `FileNotFoundError`?

```
try:
    with open('missing.txt', 'r') as f:
        data = f.read()
except FileNotFoundError:
    print('File does not exist')
```

B5 — Tricky/Conceptual Questions Teachers Love

Q: What is the output of: print(type(True))?

A: Output: . In Python, bool is a subclass of int. True == 1 and False == 0.

Q: What is the difference between deep copy and shallow copy?

A: Shallow copy (copy.copy): copies the object but not nested objects — they still share references. Deep copy (copy.deepcopy): copies the object AND all nested objects — fully independent.

Q: What is a generator? How is it different from a list?

A: A generator produces values lazily one at a time using yield — doesn't store all values in memory. A list stores all values. Generators are memory-efficient for large data.

```
def count_up(n):  
    for i in range(n): yield i
```

Q: What is the difference between pass, continue, and break?

A: pass: does nothing, placeholder for empty blocks. continue: skips the rest of the current loop iteration and goes to the next. break: immediately exits the loop entirely.

Q: What will happen: a = [1,2,3]; b = a; b.append(4); print(a)?

A: Output: [1, 2, 3, 4]. Because b = a makes b point to the SAME list object, not a copy. Modifying b also modifies a. Use b = a.copy() for independence.

Q: What is the difference between a set and a list?

A: Set: unordered, no duplicate elements, faster lookup (O(1) average), uses {}. List: ordered, allows duplicates, O(n) lookup, uses [].

Q: What does enumerate() do?

A: Returns an iterator of (index, value) pairs. Useful when you need both the index and the value in a loop.

```
for i, val in enumerate(['a','b','c']):  
    print(i, val) # 0 a, 1 b, 2 c
```

Quick Revision Cheat Sheet

Concept	Key Points to Remember
Data Types	int, float, str, bool, list, tuple, set, dict, NoneType
List vs Tuple	List=mutable [], Tuple=immutable ()
Dict key rule	Keys must be immutable (str, int, tuple OK; list NOT OK)
Slicing	a[start:stop:step] — negative step reverses
String immutability	Strings cannot be changed in-place — create new strings
File handling	Always use with open() for auto-close
Exception handling	try → except → else → finally
Sorting	Merge $O(n \log n)$, Selection $O(n^2)$, Insertion $O(n^2)$ best $O(n)$
GCD/LCM	$LCM(a,b) = a*b // GCD(a,b)$
Lambda	lambda x: x**2 — anonymous one-line function
*args/**kwargs	*args = positional tuple, **kwargs = keyword dict

r Your Viva! Study the code, understand the logic, and speak confidently.